



Presenter 4 — Live demo (part B), results & conclusion

Slides live scenarios 5, 6, 8 + full run; slides 13 – 16 · target time ≈ 9 minutes (4 min demo, 5 min slides) + leading Q&A;

Your name: _____ Roll no.: _____

You take over from: Presenter 3 (demo part A) · You hand over to: the jury (questions)

How to use this script

The **Say** text is written to be spoken at a natural pace (about 130 words a minute). You do not need to read it word for word: learn the first and last sentence of each slide and the numbers, and speak the middle in your own words. *Italic cues* tell you what to point at. Everything technical here matches the code in the repository.

The whole talk at a glance

Part	Presenter	Slides	Time	What it covers
1	Presenter 1	1 – 7	≈ 8 min	Title, abstract, introduction, problem & objectives, literature review (2 slides), research gap
2	Presenter 2	8 – 11	≈ 8 min	System architecture, install flow, attestation & logging flow, cryptography & the Warp log
3	Presenter 3	12 + live demo A	≈ 9 min	Heddle sandbox & Weave policy engine; live scenarios 1, 2, 3, 4, 7
4	Presenter 4	live demo B + 13 – 16	≈ 9 min	Live scenarios 5, 6, 8 and the full-run console; results, demo recap, conclusion, references, Q&A;

Live demonstration, part B

You continue in the terminal. Same routine: run one scenario, talk over the output, then press **Ctrl-C**.

Scenario 5 — force-push plus a planted pacman hook (defence in depth)

```
LOOM_HOLD=1 demo/run.sh forcepush
```

What you will see

- forcepush-lib 3.1 installs. Then *published forcepush-lib 3.1-1-rewrite by frank (history rewritten)*.
- **BLOCKED (pre-build)**: [FAIL] continuity, *recipe history rewritten (force-push): commit ... does not descend from it*.
- The user overrides the continuity alarm: **[OVRD]** continuity ... *accepted by override #1*, and still **BLOCKED (pre-install)**: [FAIL] placement, */usr/share/libalpm/hooks/zz-fp.hook — pacman hook: runs as root on every future package transaction*.

Say

“This models the TeamPCP incident, where more than a hundred tags were force-pushed. Loom proves the new commit does not descend from the one you installed, using git's ancestry check, so the history was rewritten. Blocked.



Now the interesting part: suppose the user reviews it and overrides that alarm. The package still carries a pacman hook, a file that would run as root on every future install. The **placement policy**, which looks at what a package installs, blocks it anyway. That is defence in depth, live.”

Scenario 6 — a Python .pth startup hook (the LiteLLM vector)

```
LOOM_HOLD=1 demo/run.sh placement
```

What you will see

- **pth-inject 1.0-1 — BLOCKED (pre-install):** [FAIL] installed files do not create persistence or privilege, `.../site-packages/loomdemo.pth` — .pth file with import lines: executes on every Python start (LiteLLM-style).

Say

“A .pth file in site-packages runs on every Python start: a perfect persistence hook that no build sandbox can see, because nothing malicious happens at build time. That is how the LiteLLM compromise worked. Loom inspects the artifact's files before installing and blocks it.”

Scenario 8 — a split-view attack on the log

```
LOOM_HOLD=1 demo/run.sh splitview
```

What you will see

- Warp restarts in split-view mode with witness-3 corrupt. The victim syncs: size 6, cosigned by all three.
- The operator forks the victim's view; thread-a and thread-b attest a new hello-loom commit to the public log; the compromised thread-c's **backdoored** attestation goes only into the fork.
- **victim re-syncs:** *transparency log verification failed: checkpoint (size 7) carries 1 valid witness cosignature(s) [witness-3]; policy requires 2 of 3.*
- An uninvolved client still sees the honest history (size 8, all three witnesses).

Say

“Now the attacker is the log operator itself, colluding with one compromised rebuilder and one corrupt witness. It shows the victim a private history containing a backdoored attestation. But the honest witnesses have already cosigned the public history, and a fork does not extend it, so they refuse. The victim's client sees only one cosignature where it needs two, and **rejects the fork**. Everyone else is unaffected.”

Finale — the full run and the live console (optional, about 70 seconds)

```
LOOM_HOLD=1 demo/run.sh
```

What you will see

- All nine scenarios stream past in the terminal. Switch to the browser tab **<http://127.0.0.1:7790>**.
- The console: service health (witness-3 turns red as corrupt in scenario 8), the log drawn as coloured threads, and the defence matrix filling in, with red cells spread across every column.

Say

“This is our deployment console, reading the same services live. Watch the defence matrix: each red cell is a mechanism stopping a package, and the red is spread across every column. No single defence does all the work. At the end the run saves this page as a report file, so the result can be inspected afterwards.”



If it misbehaves: if time is short, skip this and open [docs/demo-report.html](#) instead. It shows the same console from a completed run.

Press Ctrl-C and return to the slides at slide 13.

Slide 13 · Results: every checked acceptance criterion passes (1

min 30 s)

On screen: table, then chart, then the four tiles.

Say

Our specification defines six experiments. **E1**, attack replay: all nine malicious cases in our labelled incident corpus are blocked, and blocked **before any payload could run**. **E2**, false positives: none of the seven benign packages is blocked, even though they deliberately stress each mechanism, such as a fresh security release or a reviewed change of maintainer.

E3, build compatibility, is a measurement rather than a pass or fail gate; the live demo measures it.

E4 is the chart: the ablation study. Detections are spread across five mechanisms, and the best single one catches only three of nine, so no single bypass defeats Loom.

E5: a full policy decision takes 8.3 microseconds and verifying an Ed25519 attestation 60 microseconds in a release build, thousands of times under the targets, so installation time is dominated by the network. **E6**: 200 of 200 split views detected. And across the bottom, the sandbox contrast you saw live: zero hits confined, one unconfined.

Slide 14 · Live demonstration recap (45 s)

Say

This table recaps what you just saw live, straight from the run's event journal. Rows are the nine scenarios and columns the mechanisms. Red cells show where a mechanism stopped a package. They appear in every column, so the ablation result also holds on the live system. Scenario 8's only red cell is the witnesses column: the fork was rejected.

Slide 15 · Conclusion (1 min 30 s)

Say

To conclude: Loom turns installing community software into a verifiable decision instead of an act of faith. It **contains** every build, **verifies** artifacts through independent rebuilders and a witnessed log, **detects** changes of authority, fresh releases and persistence hooks, and **explains** every decision.

We are candid about the limitations. Rebuilder independence is simulated on one machine; a real deployment needs independent operators. Attestation proves that an artifact matches its source, not that the source is benign. And admission to the log is permissioned. Future work follows from these: a real federation of rebuilders and witnesses; npm, PyPI and crates.io backends, which the engines already support unchanged; a peer-to-peer transport; and formal verification of the policy engine.

Thank you. We are happy to take your questions.



Slide 16 · References (leave on screen)

Say

(Do not read. Leave this slide up during questions. If asked about a source, the numbers match the literature review slides.)

Leading the Q&A

Repeat each question briefly so everyone hears it, then route it: motivation and literature → Presenter 1; architecture and cryptography → Presenter 2; sandbox and policy → Presenter 3; demo, results and limitations → you.



Questions the jury is likely to ask you

Answer in two or three sentences, then stop. If a question belongs to another presenter's area, say "My teammate covered that, [name] can answer" and hand it over.

Q. Is the evaluation realistic? You built the corpus yourselves.

The corpus models the real 2024–2026 incident classes (orphan adoption, force-push, build-time injection, credential theft, worms, tampered artifacts, fresh malicious releases, pacman hooks and .pth hooks). The live fixtures are harmless probes that behave like those payloads. We acknowledge that a larger, independent corpus is future work, and E2 guards against over-fitting with deliberately tricky benign cases.

Q. What happens to packages that are not reproducible?

They never reach k attestations. By default (on_insufficient_attestations = warn) Loom then builds locally inside Heddle and warns; a stricter policy can block instead. Non-reproducibility is also measured and classified (E3).

Q. Are the performance numbers from a debug or release build?

Release: 8.3 μ s per decision and 60 μ s per Ed25519 verification, averaged over 2,000 iterations. The debug build is slower (about 9 ms per verification) but still well under the 100 ms target.

Q. Would Loom have stopped the XZ Utils backdoor?

Honestly, not on its own. XZ's payload was in the published source tarball, so every honest rebuilder would reproduce it; that is our stated out-of-scope case OOS-5. What Loom adds is the witnessed log and the audit: once the backdoor is discovered, one revocation reaches every client, and *loom audit* shows exactly which machines installed it. The sandbox would also have stopped any build-time credential theft.

Q. Doesn't failing closed make Loom annoying to use?

Every block states the rule, the evidence and the exact command to proceed (NFR-USE-2). Overrides are explicit, recorded and listed by loom audit, so the user stays in control and nothing is silently weakened.

Q. What would a real deployment need?

Independent organisations running rebuilders with diverse toolchains, several independent witnesses, an admission policy for rebuilders, and a peer-to-peer transport. The protocol and threshold logic are already real; only the deployment is simulated.

Q. Why is the demo on loopback? Is it really distributed?

Every component is a separate process talking over HTTP with its own keys: the mock AUR, the log, three witnesses, three rebuilders and three peers. Loopback just keeps the demo self-contained and safe; the same binaries run across machines by changing the addresses in the configuration.

Numbers every presenter must know

Fact	Value	Where it comes from
Malicious cases blocked (AC-1)	9 / 9 (100%)	loom-eval E1, labelled incident corpus
Benign packages blocked (AC-2)	0 / 7 (0%)	loom-eval E2
Best single mechanism (AC-4)	catches 3 of 9 (sandbox)	loom-eval E4 ablation
Policy decision time	8.3 μ s (target < 500 ms)	loom-eval E5, release build
Ed25519 attestation verification	60 μ s (target < 100 ms)	loom-eval E5, release build
Split views detected (AC-6)	200 / 200	loom-eval E6
Sink hits, same malicious build	0 confined vs 1 unconfined	demo scenario 7



Fact	Value	Where it comes from
Code base	11 Rust crates, ~12,700 lines, 75 automated tests	cargo test --workspace
Default policy	k = 2 attestations, 2-of-3 witnesses, 72 h quarantine, continuity change = block	SRS Appendix A
Deployment in the demo	mock AUR :7700, Warp :7710, witnesses :7721-3, rebuilders/peers :7731-3, console :7790	demo/run.sh